CSA0307 – Assessment 1

Arrays in C

Name: Rahul K | Reg No: 1925260036D

Part A: What is a Multidimensional Array? (5 Marks)

Definition

A multidimensional array is an array of arrays that stores data in tabular or grid format across two or more dimensions. A 1D array is a single sequence of elements, a 2D array has rows and columns, and 3D or higher arrays add further depth axes.

Syntax and Declaration in C

```
data_type array_name[dimension_1][dimension_2]...[dimension_N];
```

2D declaration example: **int matrix[3][3];** – a 3x3 matrix (3 rows, 3 columns).

Memory Layout

Although visualized as a table or cube, computer memory is linear. Multidimensional arrays are stored in contiguous memory locations using Row-Major Order, where the elements of each row are stored one after another.

Code Example

```
#include <stdio.h>

int main() {
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };
    printf("Element at row 1, column 2: %d\n", matrix[1][2]); // Output: 6
    return 0;
}
```

Output

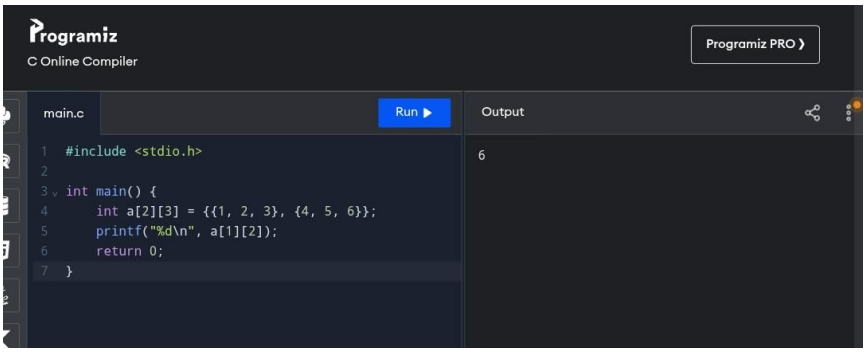


Figure 1: Output of the 2D array program (prints 6)

Part B: Explain Array Operations with Example (10 Marks)

An array supports several fundamental operations, summarized below.

Operation	Description	Time Complexity
Traversal	Visit each element once to process or display it	O(n)
Insertion	Add an element at an index; later elements shift right	O(n)
Deletion	Remove the element at an index; later elements shift left	O(n)
Searching	Locate an element (e.g., linear search)	O(n)
Updating	Change the value at a given index	O(1)

Explanation

- **Traversal:** Visiting each element exactly once to process or display it. Time complexity O(n).
- **Insertion:** Adding a new element at a specific index, which requires later elements to shift to the right. O(n) in the worst and average case.
- **Deletion:** Removing the element at a specific index and shifting later elements to the left. O(n) in the worst and average case.
- **Searching:** Locating an element in the array, e.g., by linear search. O(n) for unsorted arrays.
- **Updating:** Modifying the value stored at a specific index. O(1) by direct index access.

Complete Demonstration Program

```
#include <stdio.h>

#define MAX_SIZE 100

void traverse(int arr[], int size) {
    printf("Array elements: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int insertElement(int arr[], int *size, int element, int pos) {
    if (*size >= MAX_SIZE || pos < 0 || pos > *size) {
        return 0; // Insertion failed
    }
    for (int i = *size; i > pos; i--) {
        arr[i] = arr[i - 1];
    }
```

```
    arr[pos] = element;
    (*size)++;
    return 1;
}

int deleteElement(int arr[], int *size, int pos) {
    if (pos < 0 || pos >= *size) {
        return 0; // Deletion failed
    }
    for (int i = pos; i < *size - 1; i++) {
        arr[i] = arr[i + 1];
    }
    (*size)--;
    return 1;
}

int searchElement(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) return i; // Return index
    }
    return -1; // Not found
}

int main() {
    int arr[MAX_SIZE] = {10, 20, 30, 40, 50};
    int size = 5;

    // 1. Traversal
    traverse(arr, size);

    // 2. Insertion (Insert 25 at index 2)
    insertElement(arr, &size, 25, 2);
    printf("After inserting 25 at index 2:\n");
    traverse(arr, size);

    // 3. Deletion (Delete element at index 3)
    deleteElement(arr, &size, 3);
    printf("After deleting element at index 3:\n");
    traverse(arr, size);

    // 4. Searching
    int searchIdx = searchElement(arr, size, 40);
    if (searchIdx != -1)
        printf("Element 40 found at index %d\n", searchIdx);
    else
        printf("Element 40 not found\n");

    // 5. Update
    arr[0] = 99;
    printf("After updating index 0 to 99:\n");
    traverse(arr, size);

    return 0;
}
```

Output

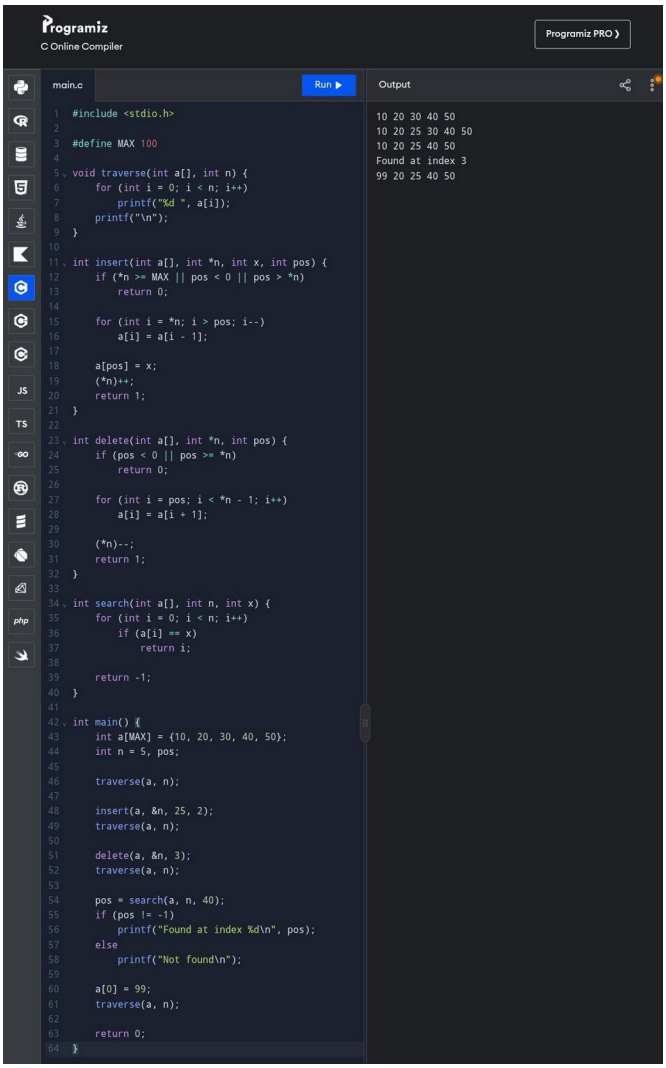


Figure 2: Program and output for the array operations

Part C: C Program to Find Frequency of Occurrence (5 Marks)

The program reads an array and counts how many times each element appears. The freq[] array is first set to -1. For each element, later matches are counted and those duplicates are marked 0 so they are not counted again. Finally, each distinct element is printed with its count.

Program

```
#include <stdio.h>

int main() {
    int arr[100], freq[100];
    int size, i, j, count;

    printf("Enter size of array: ");
    scanf("%d", &size);

    printf("Enter %d elements:\n", size);
    for (i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
        freq[i] = -1; // Initialize frequency array with -1
    }

    // Count frequency of each element
    for (i = 0; i < size; i++) {
        count = 1;
        for (j = i + 1; j < size; j++) {
            if (arr[i] == arr[j]) {
                count++;
                freq[j] = 0; // Mark duplicate element as processed
            }
        }
        if (freq[i] != 0) {
            freq[i] = count;
        }
    }

    // Print frequency result
    printf("\nFrequency of all elements in array:\n");
    for (i = 0; i < size; i++) {
        if (freq[i] != 0) {
            printf("%d occurs %d times\n", arr[i], freq[i]);
        }
    }

    return 0;
}
```

Output

Sample input: 7 elements – 10 20 10 30 20 10 40

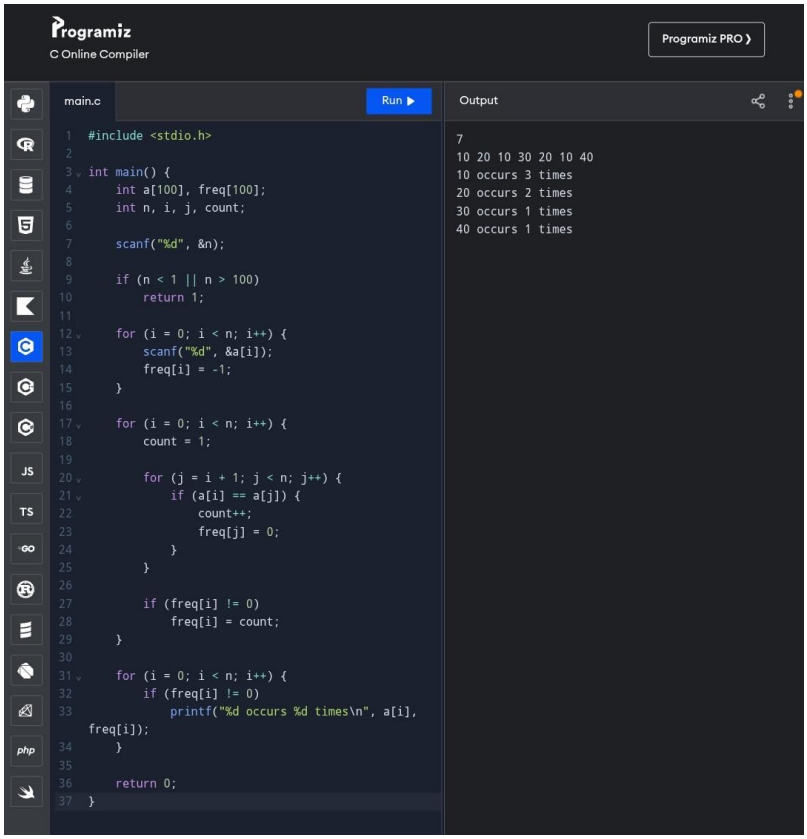


Figure 3: Program and output for frequency of occurrence